



ACTA GENERAL DEL PROYECTO

MedApp - Sistema de Gestion de Citas Medicas

Documento de consolidacion del proyecto completo elaborado a partir de la implementacion actual, la documentacion funcional y la documentacion tecnica disponibles en el repositorio.

Nombre del proyecto	MedApp - Sistema web para la gestion integral de citas medicas, pacientes, medicos, pagos, reportes y procesos de apoyo clinico.
Tipo de documento	Acta general del proyecto completo
Fecha de elaboracion	27 de abril de 2026
Estado observado del proyecto	Implementacion funcional desarrollada en backend Flask y frontend React, con integraciones tecnicas activas o previstas para Supabase, Stripe, SMTP y Twilio.
Cliente / institucion beneficiaria	Cualquier institución de salud publica o privada.

Equipo responsable	Pamela Reding
---------------------------	---------------

Acta del Proyecto MedApp

Consolidado funcional, tecnico y administrativo del sistema revisado.

1. Proposito del acta

La presente acta tiene como finalidad dejar constancia del alcance, componentes, estado funcional, arquitectura tecnica, entregables, riesgos y recomendaciones del proyecto **MedApp**, concebido como una plataforma web para la gestion de procesos medicos y administrativos asociados a la atencion de pacientes. Este documento resume el proyecto completo sobre la base de la evidencia observada en el codigo fuente, el manual de usuario y el manual tecnico disponibles en el repositorio.

2. Resumen ejecutivo

MedApp es un sistema web orientado a la administracion de citas medicas y procesos relacionados con la operacion de una consulta o centro de salud. La solucion integra modulos de autenticacion, dashboard, agenda, medicos, pacientes, recetas, pagos, reportes, horarios, notificaciones, configuracion y auditoria. El backend se encuentra desarrollado con Flask y Python, mientras que el frontend utiliza React con Vite y Tailwind CSS. La persistencia se apoya en Supabase mediante acceso HTTP/PostgREST, y existen integraciones tecnicas previstas o implementadas para correo, SMS y cobros online con Stripe.

Conclusión de estado: Se observa un sistema **funcional y estructurado por modulos**, con una base adecuada para operacion academica, demostrativa o de preproduccion, aunque todavia presenta oportunidades de fortalecimiento en pruebas automatizadas, seguridad criptografica y madurez operativa para un entorno productivo.

3. Datos generales del proyecto

Elemento	Descripcion
Nombre	MedApp - Sistema de Gestion de Citas Medicas
Dominio funcional	Salud / Gestion clinica y administrativa
Modalidad	Aplicacion web con backend y frontend desacoplados
Usuarios objetivo	Administradores, personal operativo, medicos y pacientes
Entorno de desarrollo observado	Python 3.11+, Node.js 18+, npm, Supabase, Stripe, SMTP, Twilio
Estado del producto	Implementacion funcional con documentacion de usuario y documentacion tecnica disponibles

4. Objetivo general

Desarrollar una plataforma digital integral que permita gestionar de manera centralizada las citas medicas, la informacion de pacientes y medicos, la emision de recetas, la administracion de pagos, los reportes operativos y las notificaciones del sistema, mejorando la organizacion de la operacion clinica y la trazabilidad de los procesos.

5. Objetivos especificos

1. Facilitar el registro, consulta y actualizacion de pacientes y medicos.
2. Automatizar la planificacion, confirmacion, cancelacion y reagendamiento de citas.
3. Permitir el control de horarios medicos y disponibilidad de turnos.
4. Soportar la emision y consulta de recetas medicas.
5. Registrar pagos manuales y pagos online asociados a consultas.
6. Generar reportes operativos y financieros para apoyar la toma de decisiones.
7. Ofrecer mecanismos de notificacion y recordatorio para eventos relevantes del sistema.
8. Registrar eventos de auditoria para mejorar la trazabilidad y el control administrativo.

6. Alcance del proyecto

El alcance funcional observado en la implementacion cubre los siguientes procesos:

- Acceso de usuarios al sistema mediante autenticacion y manejo de sesion.
- Registro de usuarios y recuperacion de contrasena mediante codigo enviado por correo.
- Administracion del directorio de medicos y sus datos profesionales.
- Administracion del expediente basico y clinico de pacientes.
- Gestión de agenda medica con validaciones de disponibilidad y solapamientos.
- Emision y consulta de recetas medicas.
- Creacion, consulta, cobro, reembolso y recibos de pagos.
- Visualizacion de indicadores y reportes exportables.
- Configuracion de horarios medicos, bloqueos y espacios disponibles.
- Gestión de notificaciones y consulta de eventos de auditoria.
- Configuracion de perfil, contrasena y parametros globales.

Alcance excluido o no confirmado desde la evidencia revisada: No se identifico una suite formal de pruebas automatizadas, un esquema SQL oficial versionado dentro de la copia revisada, ni artefactos de despliegue productivo completamente documentados.

7. Roles del sistema

Rol	Descripcion de alcance
admin	Acceso total a configuracion, reportes, auditoria y administracion general del sistema.
doctor	Consulta de agenda propia, seguimiento de pacientes, recetas, horarios y notificaciones.
paciente	Consulta de citas, pagos, recetas y datos personales relacionados con su atencion.

8. Modulos funcionales del sistema

Modulo	Descripcion funcional
Autenticacion	Login, registro, consulta de usuario actual, cierre de sesion y recuperacion de contrasena.
Dashboard	Panel inicial con metricas, resumen de actividad y accesos directos.
Agenda	Calendario de citas con alta, edicion, cancelacion, reagendamiento y consulta de espacios.
Medicos	CRUD de medicos, especialidades, tarifas, disponibilidad y datos profesionales.
Pacientes	Expediente general, historial clinico, signos vitales, documentos y notas por cita.
Recetas	Creacion y consulta de recetas medicas con detalle de medicamentos e instrucciones.
Pagos	Registro de cobros, pagos pendientes, pago online, reembolsos y recibos.
Reportes	Indicadores financieros y operativos con exportacion a Excel y PDF.
Horarios	Definicion de agenda base, recesos, bloqueos, feriados y duracion de slots.
Notificaciones	Consulta de eventos del sistema, recordatorios y marcado de notificaciones como leidas.
Configuracion	Actualizacion de perfil, cambio de contrasena y parametros del sistema.

Auditoria	Bitacora de acciones relevantes para control y trazabilidad administrativa.
------------------	---

9. Procesos principales cubiertos

9. Registro de paciente y apertura de expediente.
10. Registro y administracion de personal medico.
11. Asignacion de cita conforme a disponibilidad del medico.
12. Atencion de consulta y actualizacion del estado de la cita.
13. Registro de notas clinicas, historial y signos vitales.
14. Emision de receta asociada al paciente.
15. Generacion y cobranza del pago correspondiente.
16. Consulta de reportes y seguimiento operativo.
17. Envio o consulta de notificaciones y recordatorios.
18. Revision de auditoria y configuraciones del sistema.

10. Arquitectura tecnica

MedApp presenta una arquitectura web de dos capas principales. El frontend, construido en React, atiende la interaccion del usuario, mientras que el backend Flask centraliza la logica de negocio, validaciones, sesiones e integraciones. La persistencia se delega a Supabase a traves de acceso HTTP/PostgREST. Asimismo, el backend integra servicios complementarios de pagos, correo, SMS y un worker de recordatorios.

Capa	Tecnologias observadas	Responsabilidad
Frontend	React 18, React Router DOM 6, Vite 5, Tailwind CSS 3	Interfaz de usuario, navegacion, formularios, dashboards y consumo de API.
Backend	Flask 3, Python 3.11+, python-dotenv	Servicios, reglas de negocio, sesion, controladores, blueprints y endpoints.
Persistencia	Supabase / PostgreSQL via REST	Almacenamiento de usuarios, medicos, pacientes, citas, pagos y demas entidades.
Integraciones	Stripe, SMTP, Twilio	Pagos online, recuperacion de contrasena, notificaciones y recordatorios.

11. Estructura general del proyecto

Ruta principal	Contenido
----------------	-----------

app/	Configuración central, factory de Flask, rutas, controladores, modelos y servicios.
frontend/src/	Páginas React, componentes, contexto de autenticación y librerías de acceso a API.
docs/	Manual de usuario, manual técnico y presente acta general del proyecto.
requirements.txt	Dependencias Python del backend.
frontend/package.json	Dependencias y scripts del frontend.
run.py	Punto de entrada principal de la aplicación.

12. Requerimientos funcionales consolidados

Codigo	Requerimiento funcional
RF-01	El sistema debe permitir autenticacion, registro y recuperacion de contrasena.
RF-02	El sistema debe permitir registrar, listar, consultar y actualizar medicos.
RF-03	El sistema debe permitir registrar, listar, consultar y actualizar pacientes.
RF-04	El sistema debe permitir crear, editar, cancelar y reagendar citas.
RF-05	El sistema debe validar disponibilidad de horarios, recesos y bloqueos del medico.
RF-06	El sistema debe permitir registrar notas clinicas, historial y signos vitales del paciente.
RF-07	El sistema debe permitir emitir y consultar recetas medicas.
RF-08	El sistema debe registrar pagos, recibos y reembolsos.
RF-09	El sistema debe soportar pagos online con proveedor configurado.
RF-10	El sistema debe generar reportes operativos y financieros exportables.
RF-11	El sistema debe gestionar notificaciones y recordatorios.
RF-12	El sistema debe registrar eventos de auditoria para trazabilidad.

13. Requerimientos no funcionales observados o recomendados

Codigo	Requerimiento no funcional
RNF-01	Uso de sesiones seguras, control de acceso por autenticacion y cookies HTTPOnly.
RNF-02	Compatibilidad con despliegue local en Windows mediante Python y Node.js.

RNF-03	Separacion de responsabilidades por capas: rutas, controladores y servicios.
RNF-04	Capacidad de integracion con servicios externos de correo, SMS y pagos.
RNF-05	Posibilidad de exportar informacion para fines administrativos y de control.
RNF-06	Se recomienda fortalecer pruebas automatizadas, logging y seguridad criptografica para produccion.

14. Entidades de datos relevantes

Entidad	Proposito
users	Usuarios del sistema con roles y datos de acceso.
doctors	Perfiles del personal medico.
patients	Expedientes de pacientes.
appointments	Citas medicas programadas.
doctor_schedules	Horarios base de atencion por medico.
doctor_unavailability	Bloqueos, indisponibilidades y excepciones del calendario.
medical_history / vital_signs / patient_documents	Informacion clinica complementaria del paciente.
prescriptions / prescription_items / medications	Recetas y catalogo de medicamentos.
payments / refunds	Control financiero de cobros y devoluciones.
notifications / audit_log / appointment_reminders	Notificaciones internas, bitacora y recordatorios.

15. Entregables identificados

- Aplicacion web backend desarrollada con Flask.
- Aplicacion frontend desarrollada con React, Vite y Tailwind CSS.
- Modulo de agenda y gestion de citas.
- Modulo de pacientes y expediente clinico basico.
- Modulo de medicos y horarios.
- Modulo de recetas medicas.
- Modulo de pagos con soporte para cobro online.
- Modulo de reportes y exportacion de informacion.

- Modulo de notificaciones, configuracion y auditoria.
- Manual de usuario.
- Manual tecnico.
- Presente acta general del proyecto.

16. Dependencias e integraciones criticas

Componente	Uso dentro del proyecto	Nivel de criticidad
Supabase	Persistencia principal de datos mediante acceso HTTP/PostgREST.	Alta
Stripe	Procesamiento de pagos online y retorno a recibo del sistema.	Media-Alta
SMTP	Envio de correos, incluyendo recuperacion de contrasena.	Media
Twilio	Envio de SMS cuando la funcionalidad esta activada.	Media
Reminder Worker	Escaneo de citas proximas y emision de recordatorios.	Media

17. Riesgos, observaciones y puntos de mejora

Codigo	Riesgo u observacion	Impacto	Recomendacion
R-01	No se identifico una suite formal de pruebas automatizadas.	Alto	Incorporar pruebas unitarias, de integracion y de humo automatizadas.
R-02	El hashing de contraseñas observado se describe con SHA-256 y salt.	Alto	Migrar a <i>bcrypt</i> o <i>argon2</i> para endurecer seguridad.
R-03	No se encontro esquema SQL oficial versionado dentro de la copia revisada.	Medio-Alto	Formalizar migraciones y documentar el modelo de base de datos.
R-04	La integracion con Stripe requiere madurez operativa adicional para produccion.	Medio	Agregar webhook de confirmacion asincronica y monitoreo de eventos.
R-05	La operacion depende de variables de entorno y servicios externos correctamente configurados.	Medio	Documentar procedimientos de despliegue, respaldo y contingencia.
R-06	La trazabilidad y el soporte pueden mejorar con logging estructurado centralizado.	Medio	Incorporar logs estructurados y alertas de operacion.

18. Valor aportado por el proyecto

La solucion MedApp aporta valor en varias dimensiones relevantes para la gestion de servicios medicos:

- Centraliza informacion operativa y clinica en una sola plataforma.
- Reduce trabajo manual en la programacion y seguimiento de citas.
- Mejora la visibilidad del estado de pacientes, pagos y reportes.
- Facilita la atencion mediante expedientes, recetas e historiales accesibles.
- Introduce trazabilidad mediante auditoria y notificaciones.
- Prepara la base para futuras mejoras de seguridad, analitica y escalabilidad.

19. Evaluacion general del estado del proyecto

Dimension	Evaluacion
-----------	------------

Complejidad funcional	Alta. El sistema cubre de forma amplia los procesos nucleares esperados para una gestión de citas médicas.
Organización del código	Buena. Se observa separación por capas y módulos tanto en backend como en frontend.
Documentación base	Disponible. Existen manual de usuario y manual técnico.
Preparación para producción	Media. Requiere fortalecimiento en pruebas, seguridad, despliegue y gobierno de datos.
Escalabilidad futura	Favorable. La modularidad observada facilita iteraciones posteriores.

20. Conclusiones

Con base en la revisión del repositorio, la documentación existente y la estructura del sistema, se concluye que **MedApp** constituye un proyecto integral y funcional para la gestión de citas médicas y procesos asociados. El sistema presenta un alcance amplio, una arquitectura razonablemente organizada y un conjunto de módulos alineados con necesidades reales de operación clínica y administrativa.

A la vez, el proyecto muestra áreas claras de fortalecimiento necesarias para un escenario de producción formal, especialmente en materia de seguridad de credenciales, pruebas automatizadas, formalización del modelo de datos, observabilidad y manejo robusto de integraciones externas. En consecuencia, el proyecto puede considerarse **apto como solución funcional consolidada**, con recomendaciones técnicas pendientes para su maduración definitiva.

21. Recomendaciones finales

19. Adoptar *bcrypt* o *argon2* para el almacenamiento de contraseñas.
20. Implementar pruebas automatizadas para los flujos críticos del sistema.
21. Versionar formalmente la base de datos mediante migraciones.
22. Agregar webhook de Stripe y controles de conciliación de pagos.
23. Fortalecer el monitoreo y logging de procesos de correo, SMS y recordatorios.
24. Documentar procedimiento de despliegue, respaldo y recuperación ante fallos.
25. Definir responsables formales, cliente beneficiario y criterios de aceptación institucional.

22. Soportes usados para elaborar esta acta

- Archivo `README.md` del proyecto.
- Archivo `docs/MANUAL_USUARIO.md`.
- Archivo `docs/MANUAL_TECNICO.md`.
- Revisión de estructura de carpetas, rutas, configuración y componentes principales del código.

23. Firmas de conformidad

Elaborado por
Nombre y firma

Revisado por
Nombre y firma

Aprobado por
Nombre y firma

Nota: El presente documento fue elaborado con base en la implementacion observada en el repositorio revisado al 27 de abril de 2026. Los datos institucionales, autores formales o firmas oficiales no fueron identificados en la copia analizada y deberan completarse si el documento se usara para una entrega institucional o contractual.